

Non-Dimensionalized Formalism

Alexander S. Carney

First, we define the dynamical matrix symbolically

```
clear
syms kappa_c kappa_y f_c f_y J phi f_d real positive
H_orig = [-kappa_c/2 - 1j * (f_c - f_d) -1j * J ; -1j * J * exp(1j * phi)
-kappa_y/2 - 1j * (f_y - f_d)]
```

H_orig =

$$\begin{pmatrix} -\frac{\kappa_c}{2} - f_c i + f_d i & -J i \\ -J e^{\phi i} i & -\frac{\kappa_y}{2} + f_d i - f_y i \end{pmatrix}$$

eig(H_orig)

ans =

$$\begin{pmatrix} -\frac{f_c i}{2} + f_d i - \frac{f_y i}{2} - \frac{\kappa_c}{4} - \frac{\kappa_y}{4} - \sigma_1 \\ -\frac{f_c i}{2} + f_d i - \frac{f_y i}{2} - \frac{\kappa_c}{4} - \frac{\kappa_y}{4} + \sigma_1 \end{pmatrix}$$

where

$$\sigma_1 = \frac{\sqrt{-4 e^{\phi i} J^2 - f_c^2 + 2 f_c f_y + f_c \kappa_c i - f_c \kappa_y i - f_y^2 - f_y \kappa_c i + f_y \kappa_y i + \frac{\kappa_c^2}{4} - \frac{\kappa_c \kappa_y}{2} + \frac{\kappa_y^2}{4}}}{2}$$

Eigenspectrum Analysis

Looking at these eigenvalues, we can see our motivation for introducing the detuning parameters Δ_f, Δ_κ . The square root term in the eigenvalues looks like a difference of squares in terms of these detuning parameters. Let's make the substitution and see what happens.

At the same time, we will nondimensionalize the model by dividing by J .

```
syms Delta_f Delta_kappa real
A_0 = subs(H_orig, [f_y, kappa_y/2], [f_c - Delta_f, kappa_c/2 - Delta_kappa])
```

A_0 =

$$\begin{pmatrix} -\frac{\kappa_c}{2} - f_c i + f_d i & -J i \\ -J e^{\phi i} i & \Delta_\kappa - \frac{\kappa_c}{2} + \Delta_f i - f_c i + f_d i \end{pmatrix}$$

A_0_over_J = expand(simplify(A_0 / J))

A_0_over_J =

$$\begin{pmatrix} -\frac{\kappa_c}{2J} - \frac{f_c i}{J} + \frac{f_d i}{J} & -i \\ -e^{\phi i} i & \frac{\Delta_\kappa}{J} - \frac{\kappa_c}{2J} + \frac{\Delta_f i}{J} - \frac{f_c i}{J} + \frac{f_d i}{J} \end{pmatrix}$$

```
syms kappa_tilde_c kappa_tilde_y f_tilde_c f_tilde_y f_tilde_d kappa_tilde_bar real
positive
```

```
syms Delta_tilde_f Delta_tilde_kappa real
```

```
A_0_tilde = subs(A_0_over_J, ...
    [kappa_c, f_c, f_d, Delta_kappa, Delta_f]/J, ...
    [kappa_tilde_c, f_tilde_c, f_tilde_d, Delta_tilde_kappa, Delta_tilde_f])
```

A_0_tilde =

$$\begin{pmatrix} -\frac{\tilde{\kappa}_c}{2} - \tilde{f}_c i + \tilde{f}_d i & -i \\ -e^{\phi i} i & \tilde{\Delta}_\kappa - \frac{\tilde{\kappa}_c}{2} + \tilde{\Delta}_f i - \tilde{f}_c i + \tilde{f}_d i \end{pmatrix}$$

Now, we see that the eigenvalues are much simpler!

```
es = eig(A_0_tilde)
```

es =

$$\begin{pmatrix} \frac{\tilde{\Delta}_f i}{2} + \frac{\tilde{\Delta}_\kappa}{2} - \tilde{f}_c i + \tilde{f}_d i - \frac{\tilde{\kappa}_c}{2} - \sigma_1 \\ \frac{\tilde{\Delta}_f i}{2} + \frac{\tilde{\Delta}_\kappa}{2} - \tilde{f}_c i + \tilde{f}_d i - \frac{\tilde{\kappa}_c}{2} + \sigma_1 \end{pmatrix}$$

where

$$\sigma_1 = \frac{\sqrt{-\tilde{\Delta}_f^2 + 2\tilde{\Delta}_f\tilde{\Delta}_\kappa i + \tilde{\Delta}_\kappa^2 - 4e^{\phi i}}}{2}$$

```
spec_split = es(2) - es(1)
```

spec_split =

$$\sqrt{-\tilde{\Delta}_f^2 + 2\tilde{\Delta}_f\tilde{\Delta}_\kappa i + \tilde{\Delta}_\kappa^2 - 4e^{\phi i}}$$

```
syms Delta_tilde_lambda
```

```
ss = subs(spec_split, spec_split, Delta_tilde_lambda)
```

ss =

$$\tilde{\Delta}_\lambda$$

Next, we solve for the Petermann factor. In the supplement, we explain the steps that we are doing here, and supply a useful reference.

% Now, can we find the Resolvent

syms z

$Rz = \text{inv}(A_{\tilde{0}} - z \cdot \text{eye}(2));$

% The residues of the Resolvent are the eigenprojectors

$P1 = \text{simplify}(-\text{series}(Rz, z, \text{es}(1), 'Order', 2) * (z - \text{es}(1)));$

$P1 = \text{subs}(P1, \text{spec_split}, \Delta_{\tilde{\lambda}})$

$P1 =$

$$\begin{pmatrix} \frac{\tilde{\Delta}_{\kappa} + \tilde{\Delta}_{\lambda} + \tilde{\Delta}_f i}{2 \tilde{\Delta}_{\lambda}} & \frac{i}{\tilde{\Delta}_{\lambda}} \\ \frac{e^{\phi i} i}{\tilde{\Delta}_{\lambda}} & -\frac{\tilde{\Delta}_{\kappa} - \tilde{\Delta}_{\lambda} + \tilde{\Delta}_f i}{2 \tilde{\Delta}_{\lambda}} \end{pmatrix}$$

$P2 = \text{simplify}(-\text{series}(Rz, z, \text{es}(2), 'Order', 2) * (z - \text{es}(2)));$

$P2 = \text{subs}(P2, \text{spec_split}, \Delta_{\tilde{\lambda}})$

$P2 =$

$$\begin{pmatrix} -\frac{\tilde{\Delta}_{\kappa} - \tilde{\Delta}_{\lambda} + \tilde{\Delta}_f i}{2 \tilde{\Delta}_{\lambda}} & -\frac{i}{\tilde{\Delta}_{\lambda}} \\ -\frac{e^{\phi i} i}{\tilde{\Delta}_{\lambda}} & \frac{\tilde{\Delta}_{\kappa} + \tilde{\Delta}_{\lambda} + \tilde{\Delta}_f i}{2 \tilde{\Delta}_{\lambda}} \end{pmatrix}$$

$\text{normP1_2} = \text{simplify}(\text{sqrt}(\text{trace}(P1' * P1)), 'Steps', 50);$

$\text{normP2_2} = \text{simplify}(\text{sqrt}(\text{trace}(P2' * P2)), 'Steps', 50);$

$K2 = 1/2 * (\text{normP1_2}^2 + \text{normP2_2}^2)$

$K2 =$

$$\frac{\tilde{\Delta}_f^2 + \tilde{\Delta}_{\kappa}^2 + |\tilde{\Delta}_{\lambda}|^2 + 4}{2 |\tilde{\Delta}_{\lambda}|^2}$$

% Line of $K2 = 1$?

$k21_line = \text{solve}(K2 == 1, [\Delta_{\tilde{f}}, \Delta_{\tilde{\kappa}}])$

Warning: Solutions are parameterized by the symbols: x. To include parameters and conditions in the solution, specify the 'ReturnConditions' value as 'true'.

Warning: Solutions are only valid under certain conditions. To include parameters and conditions in the solution, specify the 'ReturnConditions' value as 'true'.

$k21_line = \text{struct with fields:}$

$\Delta_{\tilde{f}}: [2 \times 1 \text{ sym}]$

$\Delta_{\tilde{\kappa}}: [2 \times 1 \text{ sym}]$

To find EPs, we find where the Petermann factor diverges. We solve $\tilde{\Delta}_{\lambda} \rightarrow 0$.

```
l = abs(spec_split);
s = solve(l == 0, [Delta_tilde_f Delta_tilde_kappa], 'ReturnConditions', true);
Delta_fs_are = simplify(s.Delta_tilde_f, 'Steps', 50)
```

Delta_fs_are =

$$\begin{pmatrix} 2 \sin\left(\frac{\phi}{2}\right) - 4 \cos\left(\frac{\phi}{2}\right) i \\ -2 \sin\left(\frac{\phi}{2}\right) \\ 2 \sin\left(\frac{\phi}{2}\right) \end{pmatrix}$$

Delta_kappas_are = s.Delta_tilde_kappa

Delta_kappas_are =

$$\begin{pmatrix} -2 \cos\left(\frac{\phi}{2}\right) \\ -2 \cos\left(\frac{\phi}{2}\right) \\ 2 \cos\left(\frac{\phi}{2}\right) \end{pmatrix}$$

s.conditions;

Here, we get the parameterization of the exceptional surface.

Cubic Equation Derivation - TPDs

Now, let's do transmission stuff - hailey cubic

First, we calculate $\tilde{\beta}_{ss}$.

```
syms a real positive
B = [1; 0]; C = [0 1];
ssrepsonse = abs(C * inv(A_0_tilde) * B)^2;
beta_ss = simplify(ssrepsonse, 'Steps', 50)
```

beta_ss =

$$\frac{16}{\left(-4 \tilde{f}_c^2 + 8 \tilde{f}_c \tilde{f}_d + 4 \tilde{\Delta}_f \tilde{f}_c - 4 \tilde{f}_d^2 - 4 \tilde{\Delta}_f \tilde{f}_d + \tilde{\kappa}_c^2 - 2 \tilde{\Delta}_\kappa \tilde{\kappa}_c + 4 \cos(\phi)\right)^2 + \left(4 \sin(\phi) - 4 \tilde{\Delta}_\kappa \tilde{f}_c + 4\right)}$$

We see that the steady state transmission amplitudes attain a very complicated solution. It is hard to believe at this point that a simple closed form solution for the peaks will exist...

Continuing, we look for the critical points.

```
dss = diff(ssrepsonse, f_tilde_d) == 0;
s = solve(dss, f_tilde_d);
```

Warning: Solutions are only valid under certain conditions. To include parameters and conditions in the solution, specify the 'ReturnConditions' value as 'true'.

```
sch = children(s(1));
pol = simplify(sch{1}, 'Steps', 50);
syms z
pol_col = collect(pol, z)
```

```
pol_col =
```

$$(-8e^{\phi i})z^3 + \left(-12\tilde{\Delta}_f e^{\phi i} + 24\tilde{f}_c e^{\phi i}\right)z^2 + \left(-4e^{\phi i}\tilde{\Delta}_f^2 + 24e^{\phi i}\tilde{\Delta}_f\tilde{f}_c - 4e^{\phi i}\tilde{\Delta}_\kappa^2 + 4e^{\phi i}\tilde{\Delta}_\kappa\tilde{\kappa}_c - 24e^q\right)z + \dots$$

Here, we see that the critical points are the roots of an extremely complicated cubic equation.

After some inspiration from a documentary about the first methods to solve cubic equations, we try a depressed cubic transformation. This transformation takes the cubic:

$$az^3 + bz^2 + cz + d = 0$$

and applies the substitution $z = u - b/(3a)$ to force $a = 1, b = 0$.

Let's see if this substitution can simplify the equation

```
[coeff_list, powers] = coeffs(pol_col, z, 'All');
syms u
b = coeff_list(2);
a = coeff_list(1);
z_sub = u - b/(3 * a);
HAILEYCUBIC = simplify(subs(pol_col, z, z_sub), 'Steps', 50);
HAILEYCUBIC = simplify(rewrite(HAILEYCUBIC, 'exp'), 'Steps', 50);
HAILEYCUBIC = subs(HAILEYCUBIC, cos(phi) + 1j * sin(phi), exp(1j * phi));
[hcoeff, hpow] = coeffs(HAILEYCUBIC, u, 'All');
hnorm = simplify(HAILEYCUBIC / hcoeff(1), 'Steps', 50);
[hco, hpo] = coeffs(hnorm, u, 'All');
syms kappa_bar real positive
```

Now, we have a cubic of the form:

$$f^3 + pf + q = 0, \text{ let's see if } p \text{ and } q \text{ are simpler.}$$

```
porig = hco(3)
```

```
porig =
```

$$-\frac{\tilde{\Delta}_f^2}{4} + \frac{\tilde{\Delta}_\kappa^2}{2} - \frac{\tilde{\Delta}_\kappa\tilde{\kappa}_c}{2} + \frac{\tilde{\kappa}_c^2}{4} - \cos(\phi)$$

```
qorig = hco(4)
```

```
qorig =
```

$$\frac{\tilde{\Delta}_\kappa \sin(\phi)}{2} - \frac{\tilde{\Delta}_f \tilde{\Delta}_\kappa^2}{4} - \frac{\tilde{\kappa}_c \sin(\phi)}{2} + \frac{\tilde{\Delta}_f \tilde{\Delta}_\kappa \tilde{\kappa}_c}{4}$$

```
p_raw = simplify(subs(porig, kappa_tilde_c, kappa_tilde_bar + Delta_tilde_kappa))
```

$$p_raw = -\frac{\tilde{\Delta}_f^2}{4} + \frac{\tilde{\Delta}_\kappa^2}{4} + \frac{\tilde{\kappa}_c^2}{4} - \cos(\phi)$$

```
q_raw = simplify(subs(qorig, kappa_tilde_c, kappa_tilde_bar + Delta_tilde_kappa))
```

$$q_raw = -\frac{\tilde{\kappa}_c \left(2 \sin(\phi) - \tilde{\Delta}_f \tilde{\Delta}_\kappa \right)}{4}$$

Now, how can we write p and q in terms of the KERNEL?

```
ps = simplify(subs(p_raw, real(spec_split^2)/4, real(Delta_tilde_lambda^2)/4))
```

$$ps = \frac{\tilde{\kappa}_c^2}{4} + \frac{\text{real}(\tilde{\Delta}_\lambda^2)}{4}$$

```
qs = simplify(subs(q_raw, imag(spec_split^2)/2, imag(Delta_tilde_lambda^2)/2))
```

$$qs = \frac{\tilde{\kappa}_c \text{imag}(\tilde{\Delta}_\lambda^2)}{8}$$

```
ps = simplify(subs(ps, kappa_tilde_bar, kappa_tilde_c - Delta_tilde_kappa),  
'Steps', 50)
```

$$ps = \frac{\text{real}(\tilde{\Delta}_\lambda^2)}{4} + \frac{(\tilde{\Delta}_\kappa - \tilde{\kappa}_c)^2}{4}$$

```
qs = simplify(subs(qs, kappa_tilde_bar, kappa_tilde_c - Delta_tilde_kappa),  
'Steps', 50)
```

$$qs = -\frac{\text{imag}(\tilde{\Delta}_\lambda^2) (\tilde{\Delta}_\kappa - \tilde{\kappa}_c)}{8}$$

It is extremely surprising to us that this simplified expression was hiding in the parameterization of the peak locations the whole time!

```
syms p_tilde q_tilde real
```

Trigonometric Root Equations - Vieta

Now, we show how the roots of the depressed cubic can be simplified using a trigonometric solution. We will use this parameterization in all of our future analysis to solve for the behavior of the roots!

```
assumeAlso(kappa_bar > 0)
```

```
% first, define theta
```

```
syms theta theta_s
```

```
theta_val = (1/3) * acos(((3*q_tilde)/(2*p_tilde)) * sqrt(-3/p_tilde))
```

```
theta_val =
```

$$\frac{\operatorname{acos}\left(\frac{3\sqrt{3}\tilde{q}\sqrt{-\frac{1}{\tilde{p}}}}{2\tilde{p}}\right)}{3}$$

```
t1_pre_split_p_larger_than_0 = simplify(-2 * sqrt(p_tilde / 3) * sinh( (1/3) *  
asinh( ((3*q_tilde)/(2 * p_tilde)) * sqrt(3/p_tilde) )), 'Steps', 50)
```

```
t1_pre_split_p_larger_than_0 =
```

$$-\frac{2\sqrt{3}\sqrt{\tilde{p}}\sinh\left(\frac{\operatorname{asinh}\left(\frac{3\sqrt{3}\tilde{q}\left(\frac{1}{\tilde{p}}\right)^{3/2}}{2}\right)}{3}\right)}{3}$$

```
t1_pre_split_p_less_than_0 = simplify(-2 * (abs(q_tilde)/q_tilde) * sqrt(-  
p_tilde/3) * cosh( (1/3) * acosh( ((-3*abs(q_tilde))/(2*p_tilde)) * sqrt(-3/  
p_tilde) )), 'Steps', 50)
```

```
t1_pre_split_p_less_than_0 =
```

$$-\frac{2\sqrt{3}\sqrt{-\tilde{p}}\cosh\left(\frac{\operatorname{acosh}\left(-\frac{3\sqrt{3}|\tilde{q}|\sqrt{-\frac{1}{\tilde{p}}}}{2\tilde{p}}\right)}{3}\right)|\tilde{q}|}{3\tilde{q}}$$

```
t0 = 2 * sqrt(-p_tilde/3) * cos( theta_val - (2*pi*0/3) );  
t1 = 2 * sqrt(-p_tilde/3) * cos( theta_val - (2*pi*1/3) );  
t2 = 2 * sqrt(-p_tilde/3) * cos( theta_val - (2*pi*2/3) );  
t0 = subs(t0, theta_val, theta)
```

t0 =

$$2 \cos(\theta) \sqrt{-\frac{\tilde{p}}{3}}$$

t1 = subs(t1, theta_val, theta)

t1 =

$$2 \cos\left(\theta - \frac{2\pi}{3}\right) \sqrt{-\frac{\tilde{p}}{3}}$$

t2 = subs(t2, theta_val, theta)

t2 =

$$2 \cos\left(\theta - \frac{4\pi}{3}\right) \sqrt{-\frac{\tilde{p}}{3}}$$

% Now, simplified for nu_pm

```
theta_simp = (pi/3) + (1/3) * acos(((3*q_tilde)/(2*p_tilde)) * sqrt(-3/p_tilde));
nu_minus = 2 * sqrt(-p_tilde/3) * cos( theta_simp - pi/3 )
```

nu_minus =

$$2 \cos\left(\frac{\arccos\left(\frac{3\sqrt{3}\tilde{q}\sqrt{-\frac{1}{\tilde{p}}}}{2\tilde{p}}\right)}{3}\right) \sqrt{-\frac{\tilde{p}}{3}}$$

nu_plus = 2 * sqrt(-p_tilde/3) * cos(theta_simp + pi/3)

nu_plus =

$$2 \cos\left(\frac{2\pi}{3} + \frac{\arccos\left(\frac{3\sqrt{3}\tilde{q}\sqrt{-\frac{1}{\tilde{p}}}}{2\tilde{p}}\right)}{3}\right) \sqrt{-\frac{\tilde{p}}{3}}$$

nu_minus = subs(nu_minus, theta_simp - pi/3, theta_s - pi/3)

nu_minus =

$$2 \cos\left(\theta_s - \frac{\pi}{3}\right) \sqrt{-\frac{\tilde{p}}{3}}$$

nu_plus = subs(nu_plus, theta_simp + pi/3, theta_s + pi/3)

nu_plus =

$$2 \cos\left(\theta_s + \frac{\pi}{3}\right) \sqrt{-\frac{\tilde{p}}{3}}$$

Now, get the splitting of the roots we care about: $t_2 - t_0$.

```
splitting_is = simplify(abs(t2 - t0), 'All', true);
splitting_is = simplify(splitting_is(2))
```

```
splitting_is =
```

$$2 \left| \sin\left(\theta + \frac{\pi}{3}\right) \right| \sqrt{|\tilde{p}|}$$

```
% simpl splitting
simp_splitting_is = simplify((nu_plus - nu_minus))
```

```
simp_splitting_is =
```

$$-2 \sqrt{-\tilde{p}} \sin(\theta_s)$$

Location of TPDs

TPDs occur when $p = q = 0$

First, do the case for the $\phi = 0$ TPD.

```
p_rpep = subs(p_raw, [Delta_tilde_f, phi, kappa_tilde_bar], [0 0, kappa_tilde_c -
Delta_tilde_kappa])
```

```
p_rpep =
```

$$\frac{\tilde{\Delta}_\kappa^2}{4} + \frac{(\tilde{\Delta}_\kappa - \tilde{\kappa}_c)^2}{4} - 1$$

```
q_rpep = subs(q_raw, [Delta_tilde_f, phi], [0 0])
```

```
q_rpep = 0
```

```
rptpd = solve([p_rpep == 0, q_rpep == 0], Delta_tilde_kappa, 'ReturnConditions',
true);
delta_kappa_star_rptpd = simplify(rptpd.Delta_tilde_kappa)
```

```
delta_kappa_star_rptpd =
```

$$\left(\frac{\tilde{\kappa}_c}{2} + \frac{\sqrt{8 - \tilde{\kappa}_c^2}}{2}, \frac{\tilde{\kappa}_c}{2} - \frac{\sqrt{8 - \tilde{\kappa}_c^2}}{2} \right)$$

```
rptpd.conditions
```

```
ans =
```

$$\begin{pmatrix} \tilde{\kappa}_c^2 - 8 \leq 0 \\ \tilde{\kappa}_c^2 - 8 \leq 0 \end{pmatrix}$$

This is the TPD location that we report in the Main Text for $\phi = 0$.

```
% propagation of unceratinty
```

```
syms sigma_kappa
```

```
prop_delta_kappa_star_rtpd = abs(diff(delta_kappa_star_rtpd, kappa_tilde_c) *  
sigma_kappa)
```

```
prop_delta_kappa_star_rtpd =
```

$$\begin{pmatrix} \left| \sigma_{\kappa} \left(\frac{\tilde{\kappa}_c}{2 \sqrt{8 - \tilde{\kappa}_c^2}} - \frac{1}{2} \right) \right| \\ \left| \sigma_{\kappa} \left(\frac{\tilde{\kappa}_c}{2 \sqrt{8 - \tilde{\kappa}_c^2}} + \frac{1}{2} \right) \right| \end{pmatrix}$$

Next, do the case for the $\phi = \pi$ TPD.

```
p_arpep = subs(p_raw, [Delta_tilde_kappa, phi, kappa_tilde_bar], [0 pi,  
kappa_tilde_c])
```

```
p_arpep =
```

$$-\frac{\tilde{\Delta}_f^2}{4} + \frac{\tilde{\kappa}_c^2}{4} + 1$$

```
q_arpep = subs(q_raw, [Delta_tilde_kappa, phi, kappa_tilde_bar], [0 pi,  
kappa_tilde_c])
```

```
q_arpep = 0
```

```
arptpd = solve([p_arpep == 0, q_arpep == 0], Delta_tilde_f, 'ReturnConditions',  
true);
```

```
delta_f_star_arpep = simplify(arptpd.Delta_tilde_f)
```

```
delta_f_star_arpep =
```

$$\begin{pmatrix} \sqrt{\tilde{\kappa}_c^2 + 4} \\ -\sqrt{\tilde{\kappa}_c^2 + 4} \end{pmatrix}$$

```
arptpd.conditions
```

```
ans =
```

$$\begin{pmatrix} \text{symtrue} \\ \text{symtrue} \end{pmatrix}$$

These are the TPD locations that we report in the Main Text for $\phi = \pi$.

```
% propagation of unceratinty
```

```
prop_delta_f_star_arpep = abs(diff(delta_f_star_arpep, kappa_tilde_c) * sigma_kappa)
```

```
prop_delta_f_star_arpep =
```

$$\begin{pmatrix} \frac{\tilde{\kappa}_c |\sigma_\kappa|}{\sqrt{\tilde{\kappa}_c^2 + 4}} \\ \frac{\tilde{\kappa}_c |\sigma_\kappa|}{\sqrt{\tilde{\kappa}_c^2 + 4}} \end{pmatrix}$$

Location of Instability Transitions

Instability occurs when $\text{Re}(\tilde{\Delta}_\lambda) = \bar{\kappa}$. This is equivalent to the condition of any real eigenvalue crossing 0 to become positive.

First, do the case for $\phi = 0$

```
kernel_rpep = subs(spec_split, [Delta_tilde_f, phi], [0 0]);
rpep_inst = solve(real(kernel_rpep) == kappa_tilde_c - Delta_tilde_kappa,
Delta_tilde_kappa, 'ReturnConditions', true);
rpep_inst.Delta_tilde_kappa
```

```
ans =
```

$$\begin{pmatrix} \tilde{\kappa}_c \\ \frac{\tilde{\kappa}_c^2}{2\tilde{\kappa}_c} + 4 \end{pmatrix}$$

```
assumeAlso(kappa_tilde_c > 0)
simplify(rpep_inst.conditions, 'Steps', 50)
```

```
ans =
```

$$\begin{pmatrix} \tilde{\kappa}_c^2 - 4 < 0 \\ 0 \leq \tilde{\kappa}_c^2 - 4 \wedge \tilde{\kappa}_c \neq 2 \end{pmatrix}$$

Second, do the case for $\phi = \pi$

```
kernel_arpep = subs(spec_split, [Delta_tilde_kappa, phi], [0 pi]);
arpep_inst = solve(real(kernel_arpep) == kappa_tilde_c, Delta_tilde_f,
'ReturnConditions', true)
```

```
arpep_inst = struct with fields:
    Delta_tilde_f: [2x1 sym]
    parameters: [1x0 sym]
    conditions: [2x1 sym]
```

```
inst_delta_f = simplify(arpep_inst.Delta_tilde_f, 'Steps', 50)
```

```
inst_delta_f =
```

$$\begin{pmatrix} \sqrt{4 - \tilde{\kappa}_c^2} \\ -\sqrt{4 - \tilde{\kappa}_c^2} \end{pmatrix}$$

```
arpep_inst.conditions
```

```
ans =
```

$$\begin{pmatrix} \tilde{\kappa}_c \leq 2 \\ \tilde{\kappa}_c \leq 2 \end{pmatrix}$$

These conditions mean that for $\tilde{\kappa}_c > 2$, there is no instability transition on the EP/TPD path.

Fully generalized TPD parameterization

Step 1 - Choose a phase

```
hybrid_phase = phi;
hybrid_Delta_f = unique(real(subs(Delta_fs_are, phi, hybrid_phase)))
```

```
hybrid_Delta_f =
```

$$\begin{pmatrix} -2 \sin\left(\frac{\phi}{2}\right) \\ 2 \sin\left(\frac{\phi}{2}\right) \end{pmatrix}$$

```
hybrid_Delta_kappa = unique(real(subs(Delta_kappas_are, phi, hybrid_phase)))
```

```
hybrid_Delta_kappa =
```

$$\begin{pmatrix} -2 \cos\left(\frac{\phi}{2}\right) \\ 2 \cos\left(\frac{\phi}{2}\right) \end{pmatrix}$$

Which one of these EPs is stable? Check eigenvalues

```
first_EP_instability = simplify(unique(real(subs(es, [Delta_tilde_f,
Delta_tilde_kappa, phi], [hybrid_Delta_f(1), hybrid_Delta_kappa(1),
hybrid_phase])))) > 0, 'Steps', 50)
```

```
first_EP_instability =
```

$$\begin{pmatrix} \tilde{\kappa}_c + 2 < 4 \sin\left(\frac{\phi}{4}\right)^2 \\ \tilde{\kappa}_c + 2 < 4 \sin\left(\frac{\phi}{4}\right)^2 \end{pmatrix}$$

```
second_EP_instability = simplify(unique(real(subs(es, [Delta_tilde_f,
Delta_tilde_kappa, phi], [hybrid_Delta_f(2), hybrid_Delta_kappa(2),
hybrid_phase])))) > 0)
```

second_EP_instability =

$$\begin{pmatrix} \tilde{\kappa}_c < 2 \cos\left(\frac{\phi}{2}\right) \\ \tilde{\kappa}_c < 2 \cos\left(\frac{\phi}{2}\right) \end{pmatrix}$$

This means that the lower left EP/TPD is always stable. The top right EP can be made stable with a larger $\tilde{\kappa}_c$

Now, determine the optimal path

```
%q_hybrid = subs(q_raw, [phi, kappa_bar], [hybrid_phase, kappa_c - Delta_kappa])
%p_hybrid = subs(p_raw, [phi, kappa_bar], [hybrid_phase, kappa_c - Delta_kappa])
q_hybrid = subs(q_raw, [phi, kappa_tilde_bar], [hybrid_phase, kappa_tilde_c -
Delta_tilde_kappa])
```

q_hybrid =

$$\frac{(\tilde{\Delta}_\kappa - \tilde{\kappa}_c) (2 \sin(\phi) - \tilde{\Delta}_f \tilde{\Delta}_\kappa)}{4}$$

```
p_hybrid = subs(p_raw, [phi, kappa_tilde_bar], [hybrid_phase, kappa_tilde_c -
Delta_tilde_kappa])
```

p_hybrid =

$$\frac{\tilde{\Delta}_\kappa^2}{4} - \frac{\tilde{\Delta}_f^2}{4} - \cos(\phi) + \frac{(\tilde{\Delta}_\kappa - \tilde{\kappa}_c)^2}{4}$$

```
simplify(q_hybrid == 0)
```

ans =

$$\tilde{\Delta}_\kappa = \tilde{\kappa}_c \vee 2 \sin(\phi) = \tilde{\Delta}_f \tilde{\Delta}_\kappa$$

Thus, the optimal path is a HYPERBOLA defined by $2\sin(\phi)$ (or, there's also the $\tilde{\Delta}_\kappa = \tilde{\kappa}_c$ line as well, but this is usually unstable)

Now, find where p and q are zero simultaneously -- TPDs.

```
q_0_parameterized = solve(q_hybrid == 0, [Delta_tilde_f, Delta_tilde_kappa],
'ReturnConditions', true);
% get the solution
q_0_parameterized.Delta_tilde_kappa
```

ans =

$$\begin{pmatrix} z \\ \tilde{\kappa}_c \end{pmatrix}$$

```
q_0_parameterized.Delta_tilde_f
```

```
ans =
```

$$\begin{pmatrix} \frac{2 \sin(\phi)}{z} \\ x \end{pmatrix}$$

```
q_0_parameterized.parameters
```

```
ans = (x z)
```

```
q_0_parameterized.conditions
```

```
ans =
```

$$\begin{pmatrix} z \neq 0 \\ x \in \mathbb{R} \end{pmatrix}$$

```
% Now, get the discriminant
```

```
disc_hybrid = simplify(-4 * p_hybrid^3 - 27 * q_hybrid^2)
```

```
disc_hybrid =
```

$$4 \left(\cos(\phi) + \frac{\tilde{\Delta}_f^2}{4} - \frac{\tilde{\Delta}_\kappa^2}{4} - \frac{(\tilde{\Delta}_\kappa - \tilde{\kappa}_c)^2}{4} \right)^3 - \frac{27 (\tilde{\Delta}_\kappa - \tilde{\kappa}_c)^2 (2 \sin(\phi) - \tilde{\Delta}_f \tilde{\Delta}_\kappa)^2}{16}$$

This is a complicated result, but what we really care about here is where $p = q = 0$. That is where the TPDs are.

```
hybrid_tpd = solve([disc_hybrid == 0, q_hybrid == 0], [Delta_tilde_f,
Delta_tilde_kappa], 'ReturnConditions', true)
```

```
hybrid_tpd = struct with fields:
    Delta_tilde_f: [7x1 sym]
    Delta_tilde_kappa: [7x1 sym]
    parameters: x
    conditions: [7x1 sym]
```

```
TPD_DELTA_FS = hybrid_tpd.Delta_tilde_f
```

```
TPD_DELTA_FS =
```

$$\begin{pmatrix} -\sigma_1 \\ \sigma_1 \\ -\sigma_1 \\ -\frac{\tilde{\kappa}_c^2 x + 2\tilde{\kappa}_c x^2 - 2x^3 + 4\cos(\phi)x}{2\sin(\phi)} \\ 0 \\ 0 \\ \sigma_1 \end{pmatrix}$$

where

$$\sigma_1 = \sqrt{\tilde{\kappa}_c^2 - 4\cos(\phi)}$$

```
TPD_DELTA_KAPPAS = hybrid_tpd.Delta_tilde_kappa
```

```
TPD_DELTA_KAPPAS =
```

$$\begin{pmatrix} 0 \\ \tilde{\kappa}_c \\ \tilde{\kappa}_c \\ x \\ \frac{\tilde{\kappa}_c}{2} - \frac{\sqrt{8\cos(\phi) - \tilde{\kappa}_c^2}}{2} \\ \frac{\tilde{\kappa}_c}{2} + \frac{\sqrt{8\cos(\phi) - \tilde{\kappa}_c^2}}{2} \\ 0 \end{pmatrix}$$

```
xx = hybrid_tpd.parameters
```

```
xx = x
```

```
TPD_CONDS = simplify(hybrid_tpd.conditions, 'Steps', 50)
```

```
TPD_CONDS =
```

$$\left(\begin{array}{c} 4 \cos(\phi) \leq \tilde{\kappa}_c^2 \wedge \frac{\phi}{\pi} \in \mathbb{Z} \\ 4 \cos(\phi) \leq \tilde{\kappa}_c^2 \\ 4 \cos(\phi) \leq \tilde{\kappa}_c^2 \\ \frac{\phi}{\pi} \notin \mathbb{Z} \wedge x \in \mathbb{R} \wedge 2x^4 + 4 \cos(\phi)^2 = 2 \tilde{\kappa}_c x^3 + \left(4 \cos(\phi) - \tilde{\kappa}_c^2\right) x^2 + 4 \\ \tilde{\kappa}_c^2 \leq 8 \cos(\phi) \wedge \frac{\phi}{\pi} \in \mathbb{Z} \\ \tilde{\kappa}_c^2 \leq 8 \cos(\phi) \wedge \frac{\phi}{\pi} \in \mathbb{Z} \\ 4 \cos(\phi) \leq \tilde{\kappa}_c^2 \wedge \frac{\phi}{\pi} \in \mathbb{Z} \end{array} \right)$$

We stop here to stress that **ALL TPDs** can be parameterized in terms of these 3 variables:

TPD_DELTA_FS, TPD_DELTA_KAPPAS, and TPD_CONDS.

```
%target = in(phi/sym(pi), 'integer');
%kept_conditions = hybrid_tpd.conditions(~arrayfun(@(c) all(has(c, target, 'all')),
hybrid_tpd.conditions))
%has_bad = has(hybrid_tpd.conditions, in(phi/sym(pi), 'integer'))
```

$\phi = \pi/2$ TPDs

```
quar_with_others = TPD_CONDS(4);
cc1 = children(quar_with_others);
quar = lhs(cc1{3}) - rhs(cc1{3})
```

quar =

$$4 \cos(\phi)^2 - 2 \tilde{\kappa}_c x^3 + 2 x^4 - x^2 \left(4 \cos(\phi) - \tilde{\kappa}_c^2\right) - 4$$

To find TPDs, we need to find the roots of this quartic equation.

Here, we attempt to find a closed form solution for the roots of this quartic. It is not bi-quadratic, otherwise it would have a simple closed form solution.

We were unable to find a closed form solution. If one exists, the rest of the analysis for min splitting and splitting strength could attain closed form solutions.

Our attempt:

Since the depressed cubic made our original equation simpler, maybe a depressed quartic substitution will also help here??

```
% Depressed quartic substitution
syms u_q
coeffs_vec = (coeffs(quar, xx, 'All'))
```

```
coeffs_vec =
```

$$\begin{pmatrix} 2 & -2\tilde{\kappa}_c & \tilde{\kappa}_c^2 - 4\cos(\phi) & 0 & 4\cos(\phi)^2 - 4 \end{pmatrix}$$

```
B = coeffs_vec(2);
A = coeffs_vec(1);
quar_no_A = quar/A
```

```
quar_no_A =
```

$$2\cos(\phi)^2 - \tilde{\kappa}_c x^3 + x^4 - \frac{x^2(4\cos(\phi) - \tilde{\kappa}_c^2)}{2} - 2$$

```
x_sub = u_q - B/(4*A);
quar_depressed = collect(simplify((subs(quar_no_A, xx, x_sub)), 'Steps', 50))
```

```
quar_depressed =
```

$$u_q^4 + \left(\frac{\tilde{\kappa}_c^2}{8} - 2\cos(\phi)\right)u_q^2 + \left(-\frac{\tilde{\kappa}_c(4\cos(\phi) - \tilde{\kappa}_c^2)}{4} - \frac{\tilde{\kappa}_c^3}{8}\right)u_q + 2\cos(\phi)^2 - \frac{3\tilde{\kappa}_c^4}{256} - \frac{\tilde{\kappa}_c^2(4\cos(\phi) - \tilde{\kappa}_c^2)}{32} - 2$$

```
coeffs_dep = coeffs(quar_depressed, u_q, 'All')
```

```
coeffs_dep =
```

$$\begin{pmatrix} 1 & 0 & \frac{\tilde{\kappa}_c^2}{8} - 2\cos(\phi) & -\frac{\tilde{\kappa}_c(4\cos(\phi) - \tilde{\kappa}_c^2)}{4} - \frac{\tilde{\kappa}_c^3}{8} & 2\cos(\phi)^2 - \frac{3\tilde{\kappa}_c^4}{256} - \frac{\tilde{\kappa}_c^2(4\cos(\phi) - \tilde{\kappa}_c^2)}{32} - 2 \end{pmatrix}$$

```
P = coeffs_dep(3);
Q = coeffs_dep(4);
R = coeffs_dep(5);
```

```
syms z_r t_r
```

```
res_cub = simplify(8 * z_r^3 - 4 * P * z_r^2 - 8 * R * z_r + 4 * P * R - Q ^ 2, 'Steps', 50)
```

```
res_cub =
```

$$-\frac{3\tilde{\kappa}_c^6}{512} - \frac{5\tilde{\kappa}_c^4 z_r}{32} + \frac{\tilde{\kappa}_c^4 \cos(\phi)}{32} - \frac{\tilde{\kappa}_c^2 z_r^2}{2} + \tilde{\kappa}_c^2 z_r \cos(\phi) + \tilde{\kappa}_c^2 \cos(\phi)^2 - \tilde{\kappa}_c^2 + 8z_r^3 + 8z_r^2 \cos(\phi) - 16z_r \cos(\phi)$$

```
% Substitute z_r = t_r + P/6 to depress the cubic
```

```
z_sub = t_r + P/6;
```

```
res_depressed = simplify(subs(res_cub, z_r, z_sub), 'Steps', 100);
```

```
% Simplify the depressed cubic
```

```
res_depressed_coeff = coeffs(simplify(res_depressed, 'Steps', 100), t_r, 'All')
```

```
res_depressed_coeff =
```

$$\left(8 \ 0 \ -\frac{\tilde{\kappa}_c^4}{6} + \frac{4\tilde{\kappa}_c^2 \cos(\phi)}{3} - \frac{56 \cos(\phi)^2}{3} + 16 \ -\frac{\tilde{\kappa}_c^6}{108} + \frac{\tilde{\kappa}_c^4 \cos(\phi)}{9} + \frac{2\tilde{\kappa}_c^2 \cos(\phi)^2}{9} - \frac{2\tilde{\kappa}_c^2}{3} - \frac{272 \cos(\phi)}{27} \right)$$

```
coeffs_depressed_normalized = res_depressed_coeff / res_depressed_coeff(1)
```

```
coeffs_depressed_normalized =
```

$$\left(1 \ 0 \ -\frac{\tilde{\kappa}_c^4}{48} + \frac{\tilde{\kappa}_c^2 \cos(\phi)}{6} - \frac{7 \cos(\phi)^2}{3} + 2 \ -\frac{\tilde{\kappa}_c^6}{864} + \frac{\tilde{\kappa}_c^4 \cos(\phi)}{72} + \frac{\tilde{\kappa}_c^2 \cos(\phi)^2}{36} - \frac{\tilde{\kappa}_c^2}{12} - \frac{34 \cos(\phi)^3}{27} + \frac{4 \cos}{3} \right)$$

```
p_res = coeffs_depressed_normalized(2);
q_res = coeffs_depressed_normalized(3);
% Discriminant
Delta_resolvant = (q_res/2)^2 + (p_res/3)^3;
C1 = nthroot(-q_res/2 + sqrt(Delta_resolvant), 3);
C2 = nthroot(-q_res/2 - sqrt(Delta_resolvant), 3);
t_real = simplify(C1 + C2) % real root of resolvent cubic
```

```
t_real =
```

$$\sqrt[3]{\frac{7 \cos(\phi)^2}{6} - \sigma_1 + \frac{\tilde{\kappa}_c^4}{96} - \frac{\tilde{\kappa}_c^2 \cos(\phi)}{12} - 1} + \sqrt[3]{\sigma_1 + \frac{7 \cos(\phi)^2}{6} + \frac{\tilde{\kappa}_c^4}{96} - \frac{\tilde{\kappa}_c^2 \cos(\phi)}{12} - 1}$$

where

$$\sigma_1 = \frac{|\tilde{\kappa}_c^4 - 8\tilde{\kappa}_c^2 \cos(\phi) + 112 \cos(\phi)^2 - 96|}{96}$$

```
syms a2 a1 a0 y real
z0 = t_real;
u = sqrt(2*z0 - a2);
v = a1 / (2*u);
root1 = simplify( (-u + sqrt( - (u^2 + 2*a2 + 2*v) )) / 2 , 'Steps', 250)
```

```
root1 =
```

$$\frac{\sqrt{\sigma_1 - \frac{a_1}{\sqrt{\sigma_2 - \sigma_1 - a_2}} - a_2 - \sigma_2}}{2} - \frac{\sqrt{\sigma_2 - \sigma_1 - a_2}}{2}$$

where

$$\sigma_1 = 2 \left(\frac{|\tilde{\kappa}_c^4 - 8 \tilde{\kappa}_c^2 \cos(\phi) + 112 \cos(\phi)^2 - 96|}{96} - \frac{7 \cos(\phi)^2}{6} - \frac{\tilde{\kappa}_c^4}{96} + \frac{\tilde{\kappa}_c^2 \cos(\phi)}{12} + 1 \right)^{1/3}$$

$$\sigma_2 = 2 \left(\frac{|\tilde{\kappa}_c^4 - 8 \tilde{\kappa}_c^2 \cos(\phi) + 112 \cos(\phi)^2 - 96|}{96} + \frac{7 \cos(\phi)^2}{6} + \frac{\tilde{\kappa}_c^4}{96} - \frac{\tilde{\kappa}_c^2 \cos(\phi)}{12} - 1 \right)^{1/3}$$

```
root2 = simplify( (-u - sqrt( - (u^2 + 2*a2 + 2*v) )) / 2 , 'Steps', 250)
```

root2 =

$$-\frac{\sqrt{\sigma_1 - \frac{a_1}{\sqrt{\sigma_2 - \sigma_1 - a_2}} - a_2 - \sigma_2}}{2} - \frac{\sqrt{\sigma_2 - \sigma_1 - a_2}}{2}$$

where

$$\sigma_1 = 2 \left(\frac{|\tilde{\kappa}_c^4 - 8 \tilde{\kappa}_c^2 \cos(\phi) + 112 \cos(\phi)^2 - 96|}{96} - \frac{7 \cos(\phi)^2}{6} - \frac{\tilde{\kappa}_c^4}{96} + \frac{\tilde{\kappa}_c^2 \cos(\phi)}{12} + 1 \right)^{1/3}$$

$$\sigma_2 = 2 \left(\frac{|\tilde{\kappa}_c^4 - 8 \tilde{\kappa}_c^2 \cos(\phi) + 112 \cos(\phi)^2 - 96|}{96} + \frac{7 \cos(\phi)^2}{6} + \frac{\tilde{\kappa}_c^4}{96} - \frac{\tilde{\kappa}_c^2 \cos(\phi)}{12} - 1 \right)^{1/3}$$

```
root3 = simplify( ( u + sqrt( - (u^2 + 2*a2 - 2*v) )) / 2 , 'Steps', 250)
```

root3 =

$$\frac{\sqrt{\frac{a_1}{\sqrt{\sigma_2 - \sigma_1 - a_2}} - a_2 + \sigma_1 - \sigma_2}}{2} + \frac{\sqrt{\sigma_2 - \sigma_1 - a_2}}{2}$$

where

$$\sigma_1 = 2 \left(\frac{|\tilde{\kappa}_c^4 - 8 \tilde{\kappa}_c^2 \cos(\phi) + 112 \cos(\phi)^2 - 96|}{96} - \frac{7 \cos(\phi)^2}{6} - \frac{\tilde{\kappa}_c^4}{96} + \frac{\tilde{\kappa}_c^2 \cos(\phi)}{12} + 1 \right)^{1/3}$$

$$\sigma_2 = 2 \left(\frac{|\tilde{\kappa}_c^4 - 8 \tilde{\kappa}_c^2 \cos(\phi) + 112 \cos(\phi)^2 - 96|}{96} + \frac{7 \cos(\phi)^2}{6} + \frac{\tilde{\kappa}_c^4}{96} - \frac{\tilde{\kappa}_c^2 \cos(\phi)}{12} - 1 \right)^{1/3}$$

```
root4 = simplify( ( u - sqrt( - (u^2 + 2*a2 - 2*v) ) ) / 2 , 'Steps', 250)
```

root4 =

$$\frac{\sqrt{\sigma_2 - \sigma_1 - a_2}}{2} - \frac{\sqrt{\frac{a_1}{\sqrt{\sigma_2 - \sigma_1 - a_2}} - a_2 + \sigma_1 - \sigma_2}}{2}$$

where

$$\sigma_1 = 2 \left(\frac{|\tilde{\kappa}_c^4 - 8 \tilde{\kappa}_c^2 \cos(\phi) + 112 \cos(\phi)^2 - 96|}{96} - \frac{7 \cos(\phi)^2}{6} - \frac{\tilde{\kappa}_c^4}{96} + \frac{\tilde{\kappa}_c^2 \cos(\phi)}{12} + 1 \right)^{1/3}$$

$$\sigma_2 = 2 \left(\frac{|\tilde{\kappa}_c^4 - 8 \tilde{\kappa}_c^2 \cos(\phi) + 112 \cos(\phi)^2 - 96|}{96} + \frac{7 \cos(\phi)^2}{6} + \frac{\tilde{\kappa}_c^4}{96} - \frac{\tilde{\kappa}_c^2 \cos(\phi)}{12} - 1 \right)^{1/3}$$

We gave up from here

Uncertainty Propagation

how does uncertainty in parameters propagate into the location of TPDs?

Here, we refer to $\sigma(\tilde{\kappa}_c)$ as s_{ktc} .

$$\phi = 0$$

```
syms s_ktc real
dsplit_dkc_rptpd = diff(delta_kappa_star_rptpd(2), kappa_tilde_c);
abs_error_rptpd = dsplit_dkc_rptpd * s_ktc
```

abs_error_rptpd =

$$s_{\text{ktc}} \left(\frac{\tilde{\kappa}_c}{2 \sqrt{8 - \tilde{\kappa}_c^2}} + \frac{1}{2} \right)$$

$$\phi = \pi$$

```
dsplit_dkc_arptpd = diff(delta_f_star_arpep(1), kappa_tilde_c);
abs_error_arptpd = dsplit_dkc_arptpd * s_ktc
```

abs_error_arptpd =

$$\frac{\tilde{\kappa}_c s_{\text{ktc}}}{\sqrt{\tilde{\kappa}_c^2 + 4}}$$

```
dinst_dkc_arptpd = diff(inst_delta_f(1), kappa_tilde_c)
```

dinst_dkc_arptpd =

$$-\frac{\tilde{\kappa}_c}{\sqrt{4 - \tilde{\kappa}_c^2}}$$

```
abs_error_arptpd_inst = dinst_dkc_arptpd * s_ktc
```

abs_error_arptpd_inst =

$$-\frac{\tilde{\kappa}_c s_{\text{ktc}}}{\sqrt{4 - \tilde{\kappa}_c^2}}$$

Simplified Hybrid EP - Pi/2

Here, we plug in values for ϕ and $\tilde{\kappa}_c$ to find the location of the TPDs, since we do not attain a closed form solution.

```
hybrid_phase_choice = pi/2
```

```
hybrid_phase_choice = 1.5708
```

```
kappa_c_value_choice = 2
```

```
kappa_c_value_choice = 2
```

```
HDFS = hybrid_tpd.Delta_tilde_f;
HDKS = hybrid_tpd.Delta_tilde_kappa;
HD = hybrid_tpd.conditions
```

HD =

$$\left(\begin{array}{l} \left(8 \cos(\phi) < \tilde{\kappa}_c^2 \wedge 4 \cos(\phi) \leq \tilde{\kappa}_c^2 \wedge \frac{\phi}{\pi} \in \mathbb{Z} \right) \vee \left(4 \cos(\phi) \leq \tilde{\kappa}_c^2 \wedge \tilde{\kappa}_c^2 \leq 8 \cos(\phi) \wedge \frac{\phi}{\pi} \in \mathbb{Z} \right) \\ \left(4 \cos(\phi) \leq \tilde{\kappa}_c^2 \wedge \frac{\phi}{\pi} \notin \mathbb{Z} \right) \vee \left(8 \cos(\phi) < \tilde{\kappa}_c^2 \wedge 4 \cos(\phi) \leq \tilde{\kappa}_c^2 \wedge \frac{\phi}{\pi} \in \mathbb{Z} \right) \vee \left(4 \cos(\phi) \leq \tilde{\kappa}_c^2 \wedge \tilde{\kappa}_c^2 \leq 8 \cos(\phi) \wedge \frac{\phi}{\pi} \notin \mathbb{Z} \right) \\ \left(4 \cos(\phi) \leq \tilde{\kappa}_c^2 \wedge \frac{\phi}{\pi} \notin \mathbb{Z} \right) \vee \left(8 \cos(\phi) < \tilde{\kappa}_c^2 \wedge 4 \cos(\phi) \leq \tilde{\kappa}_c^2 \wedge \frac{\phi}{\pi} \in \mathbb{Z} \right) \vee \left(4 \cos(\phi) \leq \tilde{\kappa}_c^2 \wedge \tilde{\kappa}_c^2 \leq 8 \cos(\phi) \wedge \frac{\phi}{\pi} \in \mathbb{Z} \right) \\ \left(2 x^4 = \sigma_1 \wedge \tilde{\kappa}_c^2 < 4 \cos(\phi) \wedge \frac{\phi}{\pi} \notin \mathbb{Z} \wedge x \in \mathbb{R} \right) \vee \left(2 x^4 = \sigma_1 \wedge 4 \cos(\phi) \leq \tilde{\kappa}_c^2 \wedge \frac{\phi}{\pi} \notin \mathbb{Z} \wedge x \in \mathbb{R} \right) \\ \left(\tilde{\kappa}_c^2 < 4 \cos(\phi) \wedge \tilde{\kappa}_c^2 \leq 8 \cos(\phi) \wedge \frac{\phi}{\pi} \in \mathbb{Z} \right) \vee \left(4 \cos(\phi) \leq \tilde{\kappa}_c^2 \wedge \tilde{\kappa}_c^2 \leq 8 \cos(\phi) \wedge \frac{\phi}{\pi} \in \mathbb{Z} \right) \\ \left(\tilde{\kappa}_c^2 < 4 \cos(\phi) \wedge \tilde{\kappa}_c^2 \leq 8 \cos(\phi) \wedge \frac{\phi}{\pi} \in \mathbb{Z} \right) \vee \left(4 \cos(\phi) \leq \tilde{\kappa}_c^2 \wedge \tilde{\kappa}_c^2 \leq 8 \cos(\phi) \wedge \frac{\phi}{\pi} \in \mathbb{Z} \right) \\ \left(8 \cos(\phi) < \tilde{\kappa}_c^2 \wedge 4 \cos(\phi) \leq \tilde{\kappa}_c^2 \wedge \frac{\phi}{\pi} \in \mathbb{Z} \right) \vee \left(4 \cos(\phi) \leq \tilde{\kappa}_c^2 \wedge \tilde{\kappa}_c^2 \leq 8 \cos(\phi) \wedge \frac{\phi}{\pi} \in \mathbb{Z} \right) \end{array} \right)$$

where

$$\sigma_1 = 2 \tilde{\kappa}_c x^3 + \left(4 \cos(\phi) - \tilde{\kappa}_c^2 \right) x^2 + 4 \sin(\phi)^2$$

HDFSC = simplify(subs(HDFS, phi, hybrid_phase_choice))

HDFSC =

$$\left(\begin{array}{c} -\tilde{\kappa}_c \\ \tilde{\kappa}_c \\ -\tilde{\kappa}_c \\ x \left(\tilde{\kappa}_c^2 - 2 \tilde{\kappa}_c x + 2 x^2 \right) \\ \hline 2 \\ 0 \\ 0 \\ \tilde{\kappa}_c \end{array} \right)$$

HDKSC = simplify(subs(HDKS, phi, hybrid_phase_choice))

HDKSC =

$$\begin{pmatrix} 0 \\ \tilde{\kappa}_c \\ \tilde{\kappa}_c \\ x \\ \tilde{\kappa}_c \left(\frac{1}{2} - \frac{1}{2} i \right) \\ \tilde{\kappa}_c \left(\frac{1}{2} + \frac{1}{2} i \right) \\ 0 \end{pmatrix}$$

```
HDC = simplify(subs(HD, phi, hybrid_phase_choice))
```

HDC =

$$\begin{pmatrix} \text{symfalse} \\ \text{symtrue} \\ \text{symtrue} \\ x \in \mathbb{R} \wedge \tilde{\kappa}_c^2 x^2 + 2 x^4 = 2 \tilde{\kappa}_c x^3 + 4 \\ \text{symfalse} \\ \text{symfalse} \\ \text{symfalse} \end{pmatrix}$$

The only solutions that make sense here are the ones that are **non-zero**, because the conditions for those require ϕ to be 0 or π .

```
% HARDCODED BASED ON OUTPUT
```

```
filter = [2 3 4];  
HDFSC_TRUE = simplify(HDFSC(filter), 'Steps', 50)
```

HDFSC_TRUE =

$$\begin{pmatrix} \tilde{\kappa}_c \\ -\tilde{\kappa}_c \\ x \left(\frac{\tilde{\kappa}_c^2 - 2 \tilde{\kappa}_c x + 2 x^2}{2} \right) \end{pmatrix}$$

```
HDKSC_TRUE = simplify(HDKSC(filter), 'Steps', 50)
```

HDKSC_TRUE =

$$\begin{pmatrix} \tilde{\kappa}_c \\ \tilde{\kappa}_c \\ x \end{pmatrix}$$

```
HD_COND_TRUE = simplify(HDC(filter), 'Steps', 50)
```

HD_COND_TRUE =

$$\begin{pmatrix} \text{symtrue} \\ \text{symtrue} \\ x \in \mathbb{R} \wedge \tilde{\kappa}_c^2 x^2 + 2x^4 = 2\tilde{\kappa}_c x^3 + 4 \end{pmatrix}$$

Now, try to see where the TPDs are for various values of the control parameter

The reason why this hybrid EP is a bit simpler is because we already know 2 of the solutions to the quartic equation - it's a double root at $\tilde{\Delta}_\kappa = \tilde{\kappa}_c$

Write out the quintic

```
cc = children(HD_COND_TRUE);
last = cc(3);
full_poly = lhs(last{1}{2}) - rhs(last{1}{2})
```

```
full_poly =
```

$$\tilde{\kappa}_c^2 x^2 - 2\tilde{\kappa}_c x^3 + 2x^4 - 4$$

```
candidate_tpds = solve(full_poly == 0, xx)
```

```
candidate_tpds =
```

$$\begin{pmatrix} \text{root}(\sigma_1, z, 1) \\ \text{root}(\sigma_1, z, 2) \\ \text{root}(\sigma_1, z, 3) \\ \text{root}(\sigma_1, z, 4) \end{pmatrix}$$

where

$$\sigma_1 = z^4 - \tilde{\kappa}_c z^3 + \frac{\tilde{\kappa}_c^2 z^2}{2} - 2$$

% Now, substituet in a value

```
sols = double(vpa(subs(candidate_tpds, kappa_tilde_c, kappa_c_value_choice)))
```

```
sols = 4x1 complex
-0.7130 + 0.0000i
1.3392 + 0.0000i
0.6869 - 1.2739i
0.6869 + 1.2739i
```

```
hdfsc_true_subbed = subs(HDFSC_TRUE, kappa_tilde_c, kappa_c_value_choice)
```

```
hdfsc_true_subbed =
```

$$\begin{pmatrix} 2 \\ -2 \\ \frac{x(2x^2 - 4x + 4)}{2} \end{pmatrix}$$

```
TPD_DELTA_KAPPAS = sols(imag(sols) == 0)
```

```
TPD_DELTA_KAPPAS = 2×1
-0.7130
1.3392
```

```
TPD_DELTA_FS = double(subs(hdfsc_true_subbed(3), xx, TPD_DELTA_KAPPAS))
```

```
TPD_DELTA_FS = 2×1
-2.8051
1.4934
```

EP Eigenvalue Splitting Strength

First, what is the EP eigenvalue splitting strength

1) $\phi = 0$

```
syms epsilon_tilde real positive
split_rp = subs(imag(spec_split), [Delta_tilde_f, phi], [0 0])
```

```
split_rp =
imag( $\sqrt{\tilde{\Delta}_\kappa^2 - 4}$ )
```

```
split_rpep_ep = simplify(subs(split_rp, Delta_tilde_kappa, 2 - epsilon_tilde),
'Steps', 50)
```

```
split_rpep_ep =
imag( $\sqrt{(\tilde{\epsilon} - 2)^2 - 4}$ )
```

```
series(split_rpep_ep, epsilon_tilde, 0, 'Order', 1)
```

```
ans =
2  $\sqrt{\tilde{\epsilon}}$ 
```

2) $\phi = \pi$

```
split_arp = subs(imag(spec_split), [Delta_tilde_kappa, phi], [0 pi])
```

```
split_arp =
imag( $\sqrt{4 - \tilde{\Delta}_f^2}$ )
```

```
split_arpep_ep = simplify(subs(split_arp, Delta_tilde_f, 2 + epsilon_tilde),
'Steps', 50)
```

```
split_arpep_ep =
```

$$\sqrt{(\tilde{\epsilon} + 2)^2 - 4}$$

```
series(split_arpep_ep, epsilon_tilde, 0, 'Order', 1)
```

```
ans =
```

$$2 \sqrt{\tilde{\epsilon}}$$

3) $\phi = \pi/2$.

```
syms epsilon_tilde_h real positive
Delta_tilde_kappa_par = -sqrt(sym(2)) + epsilon_tilde_h;
Delta_tilde_f_par      = 2 / Delta_tilde_kappa_par; % keeps the product 2
split_hyp = subs(imag(spec_split), ...
                 [Delta_tilde_kappa, Delta_tilde_f, phi], ...
                 [Delta_tilde_kappa_par, Delta_tilde_f_par, pi/2]); % or any phi
you need
split_hyp = simplify(split_hyp, 'Steps', 50) % clean it up
```

```
split_hyp =
```

$$\text{imag}\left(\sqrt{\frac{(\tilde{\epsilon}_h - \sqrt{2})^2 - 4}{(\tilde{\epsilon}_h - \sqrt{2})^2}}\right)$$

```
series(split_hyp, epsilon_tilde_h, 0, 'Order', 2)
```

```
ans =
```

$$2 2^{1/4} \sqrt{\tilde{\epsilon}_h}$$

Here, we see that if you project the strength onto a single axis, the splitting strength is stronger by a factor of $2^{1/4}$.

However, we correct for this by analyzing the strength along the hyperbolic $\tilde{q} = 0$ path instead.

```
% -----
% CORRECTED UNIT STEP - EPSILON IS DIRECTLY COMPARABLE NOW.
% 3) NR-EP (phi = pi/2, hyperbolic path)
% -----
% EP coordinates on the hyperbola y = 2/x
x0 = -sqrt(sym(2)); % Delta_tilde_kappa at the EP
y0 = 2/x0; % Delta_tilde_f at the EP ( = -sqrt(2) )

% Unit-tangent normalization |(1,dy/dx)| = sqrt(1 + (2/x0^2)^2 )
norm_fac = sqrt( 1 + 4/x0^4 ); % equals sqrt(2) at this EP

% Convert arc-length step g_h to the x-coordinate increment
dx = epsilon_tilde_h / norm_fac;

Delta_tilde_kappa_par = x0 + dx; % moves exactly g_h along the curve
```

```
Delta_tilde_f_par      = 2 / Delta_tilde_kappa_par; % stays on the hyperbola
```

```
split_hyp = subs(imag(spec_split), ...
    [Delta_tilde_kappa, Delta_tilde_f, phi], ...
    [Delta_tilde_kappa_par, Delta_tilde_f_par, pi/2]);
```

```
split_hyp = simplify(split_hyp, 'Steps', 50);
series(split_hyp, epsilon_tilde_h, 0, 'Order', 2)
```

```
ans =
```

$$2 \sqrt{\tilde{\varepsilon}_h}$$

Thus, for a perturbation that moves hyperbolically, the strength is equivalent to $\phi = 0, \pi$. For a perturbation that is either in $\tilde{\Delta}_\kappa$ or $\tilde{\Delta}_f$ ONLY, the strength is stronger.

TPD Splitting Strength

Here, we analyze the Puiseux series by using the series command.

```
split_full = subs(splitting_is, [p_tilde theta], [p_raw, theta_val]);
split_full = subs(split_full, [p_tilde q_tilde], [p_raw, q_raw]);

split_full_rp = simplify(subs(split_full, [Delta_tilde_f, phi, kappa_tilde_bar], [0
0, kappa_tilde_c - Delta_tilde_kappa]), 'Steps', 50);
split_full_arp = simplify(subs(split_full, [Delta_tilde_kappa,
phi, kappa_tilde_bar], [0, pi, kappa_tilde_c]), 'Steps', 50);
split_full_nr = simplify(subs(split_full, [phi, kappa_tilde_bar], [pi/2,
kappa_tilde_c - Delta_tilde_kappa]), 'Steps', 50);
```

1) $\phi = 0$

```
split_rp_tpd = simplify(subs(split_full_rp, Delta_tilde_kappa,
delta_kappa_star_rptpd(2) + epsilon_tilde), 'Steps', 50);
split_rpep_series = series(split_rp_tpd, epsilon_tilde, 0, 'Order', 1)
```

```
split_rpep_series =
```

$$\sqrt{2} \sqrt{\tilde{\varepsilon}} \left(8 - \tilde{\kappa}_c^2\right)^{1/4}$$

2) $\phi = \pi$

```
split_arp_tpd = simplify(subs(split_full_arp, Delta_tilde_f, delta_f_star_arpep(1)
+ epsilon_tilde), 'Steps', 50);
split_arpep_series = simplify(series(split_arp_tpd, epsilon_tilde, 0, 'Order', 1))
```

```
split_arpep_series =
```

$$\sqrt{2} \sqrt{\tilde{\varepsilon}} \left(\tilde{\kappa}_c^2 + 4\right)^{1/4}$$

3) $\phi = \pi/2$

We can't find the strength in closed form, because we don't have an expression for the location of the TPD. We have to do this numerically by choosing $\tilde{\kappa}_c$.

```
Delta_kappa_par = TPD_DELTA_KAPPAS(1) + epsilon_tilde_h;
Delta_f_par = TPD_DELTA_FS(1) + 2/epsilon_tilde_h;
split_nr_tpd_ex = subs(split_full_nr, [Delta_tilde_f, Delta_tilde_kappa],
[Delta_f_par, Delta_kappa_par]);
split_nr_tpd_ex_series = simplify(series(split_nr_tpd_ex, epsilon_tilde_h, 0,
'Order', 1), 'Steps', 100);
```

Minimum Resolvable Splitting

First, we solve the Vieta equations that relate the cubic to the sum and product of its roots. At an imperfect TPD (that still crosses the discriminant), two of the roots are repeated r , and one is single s

```
syms r s p_tilde q_tilde
rel1 = 2 * r + s == 0;
rel2 = r^2 + r * s + r * s == p_tilde;
rel3 = r * r * s == -q_tilde;
whatis_s = solve(rel1, s);
% NOW that we have s, sub into the other relation
rel2 = subs(rel2, s, whatis_s);
rel3 = simplify(subs(rel3, s, whatis_s));
q_is = solve(rel3, r);
% ignore the complex solutions - q is real here
q_is = q_is(1);
% distance between the roots
dist = abs(s - r);
dist = subs(dist, s, whatis_s);
optimal_path_err = simplify(subs(dist, r, q_is))
```

optimal_path_err =

$$\frac{3 \cdot 2^{2/3} |\tilde{q}|^{1/3}}{2}$$

$\phi = 0$ min splitting

```
q_rpep_err = subs(q_raw, [phi, kappa_tilde_bar], [0, kappa_tilde_c -
Delta_tilde_kappa]);
q_rpep_splitting_err = simplify(subs(q_rpep_err, Delta_tilde_kappa,
delta_kappa_star_rptpd(2)), 'Steps', 50);
% expand around Delta_f = 0, and around the splitting location
optimal_path_err_rpep_split = expand(abs(simplify(subs(optimal_path_err, q_tilde,
q_rpep_splitting_err), 'Steps', 50)))
```

optimal_path_err_rpep_split =

$$\frac{3 \cdot 2^{2/3} \left(|\tilde{\kappa}_c^2 - 4| |\tilde{\Delta}_f| \right)^{1/3}}{4}$$

This is already in terms of a coefficient times $|\tilde{\Delta}_f|^{1/3}$

$\phi = \pi$ min splitting

```
q_arpep_err = subs(q_raw, [phi, kappa_tilde_bar], [pi, kappa_tilde_c -
Delta_tilde_kappa]);
q_arpep_splitting_err = simplify(subs(q_arpep_err, Delta_tilde_f,
delta_f_star_arpep(1)), 'Steps', 50);
optimal_path_err_arpep_split = simplify(subs(optimal_path_err, q_tilde,
q_arpep_splitting_err), 'Steps', 50)
```

optimal_path_err_arpep_split =

$$\frac{3 |\tilde{\Delta}_\kappa (\tilde{\Delta}_\kappa - \tilde{\kappa}_c)|^{1/3} (\tilde{\kappa}_c^2 + 4)^{1/6}}{2}$$

```
subs(optimal_path_err_arpep_split, kappa_tilde_c, 0)
```

ans =

$$\frac{3 \cdot 4^{1/6} (\tilde{\Delta}_\kappa^2)^{1/3}}{2}$$

This case cannot be written in terms of a coefficient times $|\tilde{\Delta}_\kappa|^{1/3}$ because of the extra term at the end. So, we take a Puiseux series expansion to make it in terms of that.

```
% assumptions to make this cleaner
assumeAlso(kappa_tilde_c > 0)
% assumeAlso(Delta_tilde_kappa - kappa_tilde_c < 0)
arpep_oep_str = abs(expand(series(optimal_path_err_arpep_split, Delta_tilde_kappa,
0, 'Order', 1)))
```

arpep_oep_str =

$$\frac{3 \tilde{\kappa}_c^{1/3} (\tilde{\kappa}_c^2 + 4)^{1/6} \left(\frac{|\tilde{\Delta}_\kappa|}{|\text{sign}(\tilde{\Delta}_\kappa)| |\text{sign}(\tilde{\Delta}_\kappa - \tilde{\kappa}_c)|} \right)^{1/3}}{2}$$

Once again, this is not an exact expansion, we are truncating the higher order terms.

$\phi = \pi/2$ min splitting

```
q_nrep_err = subs(q_raw, [phi, kappa_tilde_bar], [pi/2, kappa_tilde_c -
Delta_tilde_kappa]);
```

Here, we have the same problem as for the strength. We don't have a closed form solution for the location of the TPD, so we can't find a closed form solution for the splitting around it.